
The Compression Floor: How Few-Step Decoding Collapses in Small Block-Diffusion Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Block-diffusion language models accelerate generation by predicting multiple
2 tokens per denoising step, but the limits of this compression remain poorly under-
3 stood. Therefore, we study SDAR-4B-Chat (4B parameters block diffusion model)
4 with block size **32** on a mathematical benchmark and identify a sharp **compression**
5 **floor**: the limit beyond which predicting more tokens per step causes generation
6 quality to collapse, with three major properties. First, on a **40-problem** benchmark,
7 quality does not gradually decline with increased compression; outputs instead
8 remain stable or abruptly collapse into repetition. Across pooled schedules, **60%**
9 of outputs remain largely repetition-free ($\text{repeat-4} < 0.2$), **23%** exhibit severe
10 repetition loops ($\text{repeat-4} > 0.7$), and only **9%** lie in the intermediate regime
11 (repeat-4 between **0.3** and **0.6**). Second, supervised fine-tuning (SFT) **reduces**
12 **compression tolerance**, with the effect determined by supervision density. On a
13 **100-problem** benchmark, supervising all currently masked positions (L_{all}) reduces
14 one-token-per-step accuracy from **41%** to **7–18%** across a **three-stage** curriculum,
15 whereas supervising only positions committed at each step (L_{commit}) maintains
16 **18–33%** accuracy; after the first curriculum stage, its performance is statistically
17 indistinguishable from the base model ($p = 0.24$) at the first curriculum stage.
18 Third, on-policy self-distillation fails at two necessary preconditions: **trajectory**
19 **yield** and **teacher–student divergence**. At temperature **0.9**, the student produced **0**
20 usable trajectories in **28** attempts, whereas temperature **0.3** with a larger generation
21 budget yielded **38/50**; the similar yield collapse occurs for **TraDo-4B**, suggesting it
22 is not architecture-specific. Moreover, our privileged-prefix teacher overlaps with
23 the student on **86%** of its **top-20** token set on average, indicating weak distillation
24 signal, while a within-block lookahead teacher reduces this overlap to **57%** on
25 the same denoising states. Together, these results show that few-step diffusion
26 language modeling is constrained not only by the number of tokens revealed per
27 step, but by the interaction of **sampling, supervision, and teacher construction**,
28 which must jointly provide sufficient signal to operate beyond the model’s com-
29 pression floor. The Code is available at: [https://anonymous.4open.science/](https://anonymous.4open.science/r/Compression_Floor--Diffusion_Language_Model-1BC2/README.md)
30 [r/Compression_Floor--Diffusion_Language_Model-1BC2/README.md](https://anonymous.4open.science/r/Compression_Floor--Diffusion_Language_Model-1BC2/README.md).

31 1 Introduction

32 Diffusion language models generate text through iterative denoising, progressively refining a partially
33 specified sequence while predicting multiple tokens in parallel at each step. Block-diffusion variants
34 such as SDAR [Cheng et al., 2025] and TraDo [Wang et al., 2025] extend this idea by decoding tokens
35 in fixed-size blocks in parallel while remaining autoregressive across blocks. This offers a simple
36 route to faster generation: increasing the number of tokens decoded per step reduces the number of

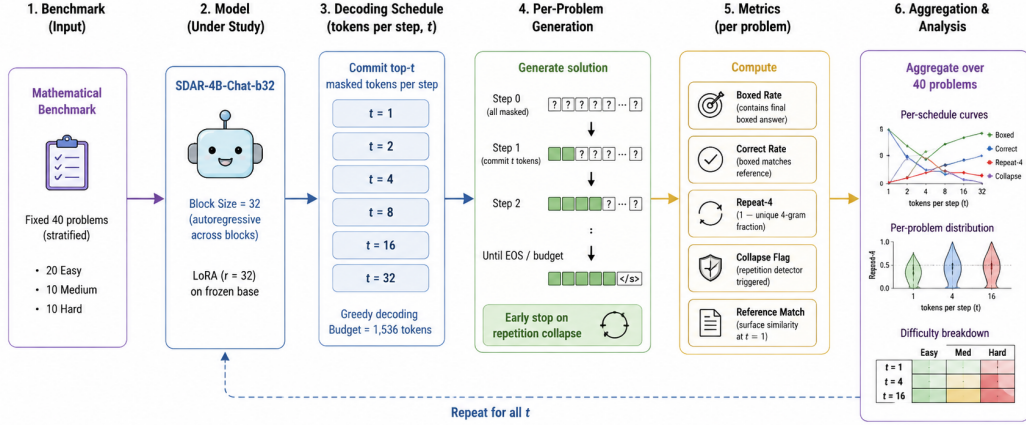


Figure 1: **The Compression Floor Evaluation Pipeline.** We evaluate SDAR-4B-Chat-b32 on a fixed mathematical benchmark across decoding schedules with $t \in \{1, 2, 4, 8, 16\}$ tokens committed per denoising step (single-step $t = 32$ schedule is shown in the pipeline but not reported). For each schedule, the model generates solutions using greedy decoding until EOS or a 1,536-token budget, with early stopping upon repetition collapse. We compute boxed rate, correct rate, repeat-4 and collapse rate for each problem, then aggregate results across schedules and difficulty levels.

forward passes required. *Understanding how far this compression can be pushed is therefore critical for determining the practical speedup and compute efficiency of diffusion language models.*

However, this speedup raises a fundamental question: *how far can we increase the number of tokens decoded per step before generation quality breaks down?* We study this question using SDAR-4B-Chat with block size 32 on mathematical reasoning problems from DeepMath-103K [He et al., 2025]. We fix a 40-problem benchmark and systematically vary the number of tokens revealed per step from 1 to 16, allowing us to characterize the model’s *compression floor*—the point beyond which more aggressive decoding causes generation to collapse. Overcoming or better understanding this floor could enable faster and more compute-efficient generation, improving the practicality of diffusion language models for large-scale reasoning and real-time applications.

We then investigate whether this floor can be shifted through training. First, we study whether supervised fine-tuning (SFT) improves tolerance to aggressive decoding and whether its effect depends on supervision density. Second, we evaluate whether on-policy self-distillation, which trains a fast student against a better-informed teacher constructed from the same model weights [Zhao et al., 2026], can recover accuracy at higher compression.

Our results reveal that the *compression floor* is not a smooth degradation that training can simply polish away. Instead, aggressive decoding produces a *sharp, bimodal failure mode*: generations tend to remain either largely coherent or to collapse into severe repetition loops. Moreover, SFT does not improve the performance during decoding, with the extent of degradation being strongly dependent on *supervision density*. These findings suggest that the achievable speedup of block-diffusion models is constrained by an intrinsic interaction between *decoding compression* and *the way the model is trained*.

Self-distillation never reaches the training stage unless the student can produce usable trajectories at all, and whether it can is decided by sampling parameters, not by the method. Once trajectories exist, the teacher must still disagree with the student enough to supply a gradient, and we show that the standard privileged-prefix teacher does not.

Our contributions are the following measurements:

- On a 40-problem benchmark, we show that the few-step failure in a small block-diffusion model is a per-sample phase transition. Individual outputs are either fluent or collapsed, and mean metrics degrade only because the collapsed fraction grows (Section 4.1).

- We isolate the effect of SFT supervision density with a three-arm ablation on a 100-problem benchmark. Supervising every masked position degrades the model at every curriculum stage, and giving it fresh data instead of a reused pool does not change this. Restricting supervision to the positions actually committed at each step preserves base accuracy at the first stage and roughly halves the accuracy loss overall, but does not restore base accuracy at later stages (Section 4.2).
- We measure both preconditions of on-policy self-distillation for diffusion models. Trajectory yield is controlled by sampling temperature and budget, and we replicate the yield collapse on a second model family. Teacher-student divergence fails for the privileged-prefix teacher we trained, and we measure that a within-block lookahead teacher has substantially more divergence on the same states (Section 4.3).

Recent work has pursued *few-step diffusion language modeling* along two complementary directions. Both masked and block-diffusion hybrids such as SDAR [Cheng et al., 2025] and TraDo [Wang et al., 2025] reduce sequential depth by revealing multiple tokens in parallel, while training-free methods such as Fast-dLLM [Wu et al., 2025] further accelerate inference without modifying the model. These approaches expose a fundamental trade-off between *generation speed and the number of denoising steps*, and recent analyses attribute few-step degradation to errors introduced when multiple tokens must be predicted simultaneously. A separate line of work fits *compression laws* to parameter pruning, deriving smooth functional forms and a critical compression ratio beyond which degradation becomes unacceptable [Sengupta et al., 2025]. Temporal compression of the denoising schedule does not admit the same treatment at the scale we study, because the per-sample distribution is bimodal rather than smoothly shifted, so a mean-level law would describe a mixture rather than a degradation curve.

A second line of work attempts to recover this lost quality through *distillation*. T3D [Zhang et al., 2026] distills on rollout trajectories produced under the target decoding procedure, while OPSD [Zhao et al., 2026] introduces *on-policy self-distillation*, in which teacher and student share weights and the teacher conditions on privileged information while both are evaluated on the student’s own rollout. However, these approaches implicitly require the student to produce sufficiently useful trajectories for training.

In contrast, we focus on the regime where this prerequisite itself breaks down: we characterize the *compression floor* at the level of individual generations and study how supervised fine-tuning and sampling choices move or expose this floor. Thus, rather than proposing another acceleration or distillation method, our work provides an empirical characterization of when few-step diffusion generation remains trainable and when it collapses.

2 Methodology

2.1 Problem formulation:

A block-diffusion language model generates $y = (y^1, \dots, y^L)$ in blocks of size B , decoding one block at a time under a causal ordering across blocks. Within each block, all positions are initially masked and are progressively revealed by committing the t highest-confidence predictions at each denoising step. Thus, larger t corresponds to a coarser schedule that commits more tokens per forward pass and reduces decoding cost, but also requires more predictions to be made independently at the same step. We study $t \in 1, 2, 4, 8, 16$ across $B \in 16, 32$ with fixed base weights p_θ . Post-training compares dense supervision over all block positions with masked-only supervision over positions that remain unrevealed at each state. Self-distillation uses teacher and student distributions from different conditioning contexts along the student’s on-policy trajectory, minimizing a step-level divergence while keeping the training signal restricted to states encountered by the student.

2.2 Formal Metric Definitions

Boxed rate and correct rate. For a generation g and reference answer a^* , let $\text{box}(g)$ denote the extracted contents of the final `\boxed{\}` expression in g , or $\perp$ if none is present. Boxed rate over a set of N problems is $\text{Boxed} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{box}(g_i) \neq \perp]$. Correct rate additionally requires an exact

117 match to the verified reference, $\text{Correct} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{box}(g_i) = a_i^*]$. Correct rate is a subset of
 118 boxed rate by construction.

119 **Repeat-4.** For a generation g tokenized into n -grams of length 4, let $U_4(g)$ be the number of distinct
 120 4-grams and $C_4(g)$ the total count of 4-grams. Repeat-4 is $\mathcal{R}_4(g) = 1 - U_4(g)/C_4(g)$. Lower values
 121 indicate greater lexical diversity; values approaching 1 indicate that the generation is dominated by a
 122 small set of repeated 4-grams.

123 **Collapse flag.** A generation is flagged as *collapsed* if the online repetition detector halts decoding
 124 before the token budget or EOS is reached. The collapse rate over N problems is the fraction so
 125 flagged, $\text{Collapse} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{halted by repetition detector}]$.

126 **Trajectory yield.** For on-policy self-distillation screening (Section 4.3), a rollout is *usable* if
 127 $\text{box}(g) \neq \perp$ within the sampling budget. Yield at a given temperature τ and budget B is
 128 $\text{Yield}(\tau, B) = \frac{1}{M} \sum_{i=1}^M \mathbb{I}[\text{box}(g_i) \neq \perp]$ over M screening attempts. Yield is a precondition
 129 for downstream distillation and is measured independently of correctness.

130 **Top- K overlap.** For a denoising state and a position the student commits at that state, let T_K^{stu} and
 131 T_K^{tea} denote the K highest-probability token sets under the student and teacher, respectively, both
 132 evaluated on the student’s own partially revealed block. The overlap is the mean over committed
 133 positions of $\frac{|T_K^{\text{stu}} \cap T_K^{\text{tea}}|}{K}$. We use $K = 20$, following Luo et al. [2026]. An overlap of 1 means that
 134 the teacher and student candidate sets are identical, so a KL term on those candidates carries no
 135 information.

136 **Dense loss and commit loss.** For a block containing L masked-position candidates at a given
 137 denoising state, let $M_t \subseteq \{1, \dots, L\}$ denote the currently masked positions at step t , and let
 138 $K_t \subseteq M_t$ denote the top- t highest-confidence positions among M_t , i.e., exactly those positions
 139 committed at that step. The two supervision objectives used in Section 4.2 differ only in which
 140 positions they sum the per-position loss $\ell(\cdot)$ over. The dense (all-position) loss supervises every
 141 currently masked position in the block,

$$L_{\text{all}} = \frac{1}{|M_t|} \sum_{i \in M_t} \ell(y^i, \hat{y}^i),$$

142 while the commit (commit-only) loss restricts supervision to the positions actually committed at that
 143 step,

$$L_{\text{commit}} = \frac{1}{|K_t|} \sum_{i \in K_t} \ell(y^i, \hat{y}^i).$$

144 Because $K_t \subseteq M_t$, L_{commit} is computed over a subset of the positions used by L_{all} ; the dense objective
 145 supervises positions that will remain masked and be revised at later steps, while the commit objective
 146 supervises only the positions that are finalized at the current step.

147 3 Experimental Setup

148 **Model and data:** We conduct our experiments on SDAR-4B-Chat [Cheng et al., 2025], a 4B-
 149 parameter block-diffusion language model with block size 32 (b32) and block size 16 (b16), obtained
 150 by adapting an autoregressive base model. Training and evaluation instances are drawn from
 151 DeepMath-103K [He et al., 2025], which provides verified final answers together with three R1-
 152 generated solution traces per problem. We use two fixed held-out benchmarks. The *40-problem*
 153 *benchmark* is stratified by dataset difficulty into 20 easy, 10 medium, and 10 hard instances and
 154 is used for the full multi-schedule decoding grid in Section 4.1. The *100-problem benchmark* is
 155 an independently drawn, disjoint set stratified into 50 easy, 25 medium, and 25 hard instances
 156 (DeepMath difficulty ≤ 4 , ≤ 6 , and > 6 respectively) and is used for the supervision-density ablation
 157 in Section 4.2. The 40-problem set was sized to keep the extensive multi-schedule decoding and
 158 training experiments computationally feasible. Unless otherwise specified, fine-tuning is performed
 159 with LoRA adapters of rank 32 while keeping the base model frozen.

160 **Decoding protocol:** For a block containing 32 masked positions, we decode t tokens per denoising
 161 step by committing the t highest-confidence masked positions. Thus, $t = 1$ requires 32 denoising
 162 steps per block, whereas $t = 32$ completes the block in a single step. We evaluate schedules spanning

$t \in \{1, 2, 4, 8, 16\}$ on the 40-problem benchmark and $t \in \{1, 2\}$ on the 100-problem benchmark, and use greedy decoding unless otherwise stated. Generations are limited to 1,536 tokens on the 40-problem benchmark and 1,024 tokens on the 100-problem benchmark.

Evaluation metrics: We measure both task completion and generation stability. *Boxed rate* is the fraction of generations containing a final boxed answer, while *correct rate* additionally requires the extracted boxed answer to match the verified reference answer. To quantify degenerative repetition, we use *Repeat-4*, defined as one minus the fraction of unique 4-grams in a generation. Thus, lower values indicate greater diversity, while values approaching one correspond to severe repetition.

Evaluation protocols: We use two evaluation harnesses for distinct experimental comparisons while keeping the prompts fixed. The decoding-grid experiments in Section 4.1 uses a common grid-based evaluator with an explicit repetition-collapse stopping criterion. The three-stage curriculum comparison in Section 4.2 uses the training harness associated with the corresponding fine-tuning runs and reports a more permissive string-match accuracy. Because the stopping and matching criteria differ, results from the two protocols are treated as separate measurements and are never combined within a single comparison. Where needed, the base model is evaluated independently under each protocol to provide the appropriate reference point.

4 Results

4.1 The floor is a per-sample phase transition

Table 1 reports the base model at block 32 across schedules. Accuracy holds up to $t = 2$, degrades at $t = 4$, and reaches zero between $t = 4$ and $t = 8$. The collapse-detector stop fraction mirrors this cliff, rising from 0 at $t \leq 2$ to 0.62 at $t = 16$. Block size 16 shows the identical cliff location (correct 0.20 at $t = 4$, 0.00 at $t = 8$).

Table 1: Base SDAR-4B-Chat at block sizes 16 and 32 across decoding schedules (40 problems per cell, greedy). Accuracy remains relatively stable at fine schedules but exhibits a sharp cliff between 4 and 8 tokens per step for both block sizes. Collapse denotes the fraction of generations halted by the repetition detector, which increases sharply beyond this transition and indicates the onset of severe repetitive degeneration.

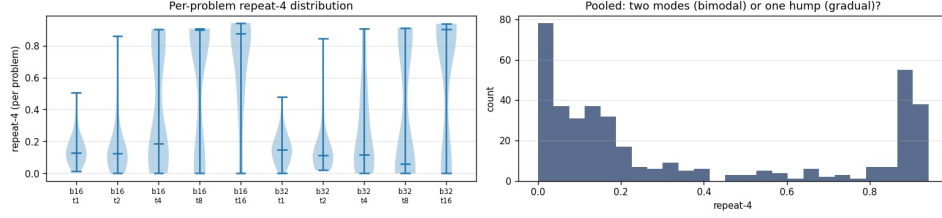
Block	Metric	$t=1$	$t=2$	$t=4$	$t=8$	$t=16$
16	correct	.425	.375	.200	.000	.000
	repeat-4	.152	.181	.403	.579	.684
	collapse	.000	.000	.100	.500	.500
32	correct	.375	.250	.150	.000	.000
	repeat-4	.161	.156	.268	.386	.590
	collapse	.000	.000	.050	.275	.625

The mean curves hide the structure. Figure 2 pools per-problem repeat-4 over all schedules. The distribution is bimodal. At block 32, 60% of outputs sit below 0.2, 23% sit above 0.7, and only 9% occupy the middle band from 0.3 to 0.6. Individual generations do not degrade gradually as the schedule coarsens. They either hold together or fall into a loop, and coarser schedules only shift mass between the two modes. This matches the factorization-error account of few-step decoding [Zhang et al., 2026]: once simultaneously committed tokens must be predicted independently, an unlucky commitment locks the block, and everything after it, into repetition. The cost is also difficulty-selective. At block 32 and $t = 1$, medium-tier boxed rate is 0.60, and it is the first tier to reach zero as t grows.

4.2 SFT lowers the accuracy, and supervision density decides how much

We fine-tuned the model on DeepMath solutions, always with LoRA, and benchmarked every checkpoint on the fixed 100 problems.

A three-arm curriculum separates the objective from the data. We ran three matched three-stage curricula (stages at 4, 2, and 1 tokens per step, 500 problems and 2 epochs per stage) and



The degradation is driven by the **objective, not the amount of data**. Giving dense supervision more fresh data does not improve its performance: Dense₁₅₀₀ and Dense₅₀₀ are statistically indistinguishable at both $t = 1$ ($p = 0.80$) and $t = 2$ ($p = 0.19$). Both dense objectives substantially reduce accuracy from the 41% base, reaching 7–18% at $t = 1$ and at most 7% at $t = 2$. In contrast, the commit-only objective performs significantly better on the same disjoint data schedule, reaching 24.0% vs. 11.3% for Dense₁₅₀₀ at $t = 1$, and 13.3% vs. 4.7% at $t = 2$ (both $p < 0.001$). This shows that **supervision density, rather than data volume, is the main factor behind the degradation**.

The commit-only objective also degrades more gradually as the curriculum becomes finer. At $t = 1$, its stage-1 checkpoint retains 33% accuracy, which is not significantly different from the 41% base accuracy ($p = 0.24$), while later stages drop to 18% and 21%. At $t = 2$, accuracy remains relatively stable across stages (14%, 14%, and 12%). This is consistent with the fact that commit-only supervision becomes sparser as the curriculum advances.

The two objectives also differ in **generation stability**. Dense supervision substantially increases repetition and collapse, whereas commit-only causes much smaller increases. At $t = 1$, repeat-4 rises from 0.157 for the base model to 0.38–0.52 under dense supervision, but only to 0.21–0.25 under commit-only. Similarly, dense checkpoints show 2–17% collapse stops, compared with at most 1% for commit-only. The same pattern appears in difficulty-specific performance: dense checkpoints are especially fragile on hard problems, while commit-only retains non-zero performance at the later stages. Overall, **dense supervision causes substantially more accuracy and generation-quality degradation than commit-only supervision, and adding more data does not fix it**.

4.3 Self-distillation fails at both preconditions

Precondition 1: Trajectory Yield. On-policy self-distillation requires the student to first generate useful trajectories; for our math tasks, we define a useful rollout as one that reaches a boxed final answer within the generation budget. Table 3 shows that this yield depends strongly on the sampling settings: with temperature 0.9 and a 768-token budget, SDAR-4B-Chat produced 0 usable rollouts from 28 attempts, whereas lowering the temperature to 0.3 and increasing the budget to 1,536 tokens yielded 38 usable rollouts from 50 attempts, including 13 correct ones. The failed rollouts were not necessarily poor generations—their mean repeat-4 was only 0.17, and none produced an incorrect boxed answer—but simply failed to reach a conclusion within the budget. Because temperature and budget changed together, their individual effects cannot be separated; however, the low repetition and absence of incorrect boxed answers suggest that the budget was the binding constraint. We observed the same issue with TraDo-4B [Wang et al., 2025], where temperature 0.9 produced no boxed rollouts in the first 36 attempts. This has implications for existing diffusion OPSD methods: Luo et al. [2026] construct the self-teacher from the student’s own on-policy trajectories, and on GSM8K-scale tasks report that a pass@ k budget of $k = 8$ outperforms $k = 1$, with $k = 1$ showing a measurable drop (Table 8 of their work). Our results suggest a possible reason: usable trajectories may be rare under some sampling settings, so increasing the number of samples alone cannot solve the problem if the generation settings prevent the model from reaching a conclusion. Thus, the required sampling budget is model- and regime-dependent, and **trajectory yield should be measured and reported before self-distillation**, since methods that rely on usable student trajectories cannot operate effectively when those trajectories are not produced in the first place.

Precondition 2: Teacher Divergence. Distillation on the 38 accepted trajectories failed because the loss collapsed to near zero within a few steps. We therefore measured teacher–student agreement using frozen base weights: the student ran at 2 tokens per step on 20 problems, giving 6,224 denoising states, and we measured top-20 overlap at committed positions (Section 2.2). On the same states, the privileged-prefix teacher, conditioned on a verified reference solution, had mean overlap 0.856, while the self-future lookahead teacher of Luo et al. [2026], which sees only the student’s prompt and next four future positions, had overlap 0.567 (Table 3; Figure 4). The gap held on all 20 problems: privileged-teacher means ranged from 0.811–0.892 versus 0.536–0.617 for lookahead, with no overlap between the ranges. Because overlap near 1 implies nearly identical top- K sets, the privileged teacher’s clipped KL target provides little information, explaining the loss collapse. Thus, at 4B scale, the privileged-prefix construction fails the teacher-divergence precondition, whereas the lookahead teacher satisfies it. This is a negative result for the privileged construction, not for on-policy self-distillation generally; whether the lookahead teacher improves accuracy remains untested.

Table 3: Screening-rollout yield and teacher–student distillation signal. **Panel A:** usable rollout yield under different sampling settings (usable = boxed final answer). **Panel B:** teacher–student top-20 overlap across 6,224 denoising states from 20 problems at 2 tokens per step (lower overlap indicates more distillation signal).

Panel A: Trajectory yield				
Model	Temp. / budget	Usable	Correct	Mean repeat-4
SDAR-4B-Chat-b32	0.9 / 768	0 / 28	0 / 28	.17
SDAR-4B-Chat-b32	0.3 / 1,536	38 / 50	13 / 50	— ^a
TraDo-4B	0.9 / 768	0 / 36	0 / 36	— ^a

^aNot logged in these runs.

Panel B: Teacher-student top-20 overlap			
Teacher construction	Mean overlap	Per-problem range	Lower on $k/20$ problems
Privileged prefix (reference solution)	0.856	0.811 to 0.892	0
Self-future lookahead (4 future positions)	0.567	0.536 to 0.617	20

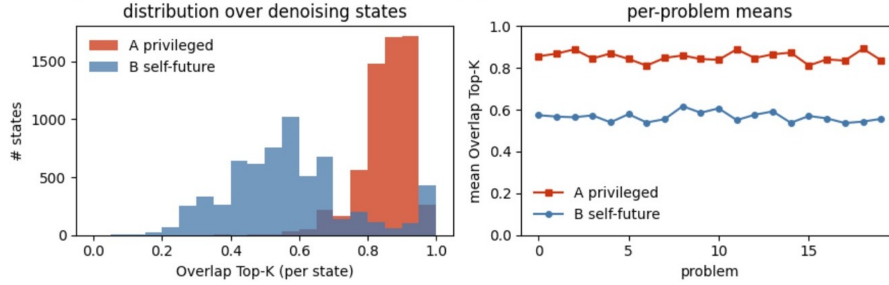


Figure 4: Top-20 overlap between teacher and student. Left: distribution over the 6,224 denoising states. Right: per-problem means. The privileged-prefix teacher sits near 0.85 on every problem; the lookahead teacher sits near 0.57.

264 5 Limitations and conclusion

265 **Limitations:** Our decoding-grid results use one 40-problem benchmark and one seed, giving binomial
 266 uncertainty of about ± 15 points, so they should be interpreted as evidence of large effects rather
 267 than precise estimates. The supervision-density ablation uses a separate 100-problem benchmark,
 268 reducing uncertainty to roughly ± 10 points at the base rate and ± 5 points after fine-tuning, but
 269 each checkpoint is still a single run with one seed. Pooled comparisons in Table 2 treat stages as
 270 independent, so per-stage tests provide the more conservative evidence. All experiments use LoRA on
 271 one 4B model, and results may differ with full fine-tuning or larger models. Many reference solutions
 272 exceed the training-length limit and are truncated without stop tokens; Figure 5 shows that budget
 273 exhaustion without a boxed answer dominates stopping for every fine-tuned checkpoint, consistent
 274 with truncation contributing to non-termination. Finally, we do not measure post-distillation accuracy:
 275 we show that the privileged-prefix teacher provides little signal, while a lookahead teacher provides
 276 substantially more, but we do not train with the latter. Our results therefore establish distillation
 277 preconditions, not its eventual benefit.

278 **Conclusion:** The compression limit of a small block-diffusion language model is not a smooth
 279 performance decline but a sharp transition from successful generation to repetitive collapse. Further-
 280 more, fine-tuning worsens the accuracy across all variants, with supervision density as the key factor:
 281 dense supervision over all masked positions degrades accuracy at every stage regardless of data reuse,
 282 whereas commit-only supervision preserves base accuracy at the coarsest stage and retains roughly
 283 twice the accuracy of dense supervision (24.0% vs 11.3% pooled). Self-distillation additionally
 284 requires two preconditions: sufficient usable-trajectory yield, which depends strongly on sampling
 285 temperature and generation budget, and teacher–student divergence, which the privileged-prefix
 286 construction lacks at this scale. Future work should therefore report collapse rates, supervision
 287 density, trajectory yield, and teacher–student divergence alongside final distillation performance.

References

- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.
- Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Bqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- Justin Deschenaux and Çağlar Gülçehre. Beyond autoregression: Fast LLMs via self-distillation through time. In *International Conference on Learning Representations (ICLR)*, 2025. *arXiv:2410.21035*.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.
- Yifu Luo, Zeyu Chen, Haoyu Wang, Xinhao Hu, Yuxuan Zhang, Zhizhou Sha, and Shiwei Liu. Learning from the self-future: On-policy self-distillation for dllms. *arXiv preprint arXiv:2606.18195*, 2026.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Jirong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. *arXiv:2406.07524*.
- Ayan Sengupta, Siddhant Chaudhary, and Tanmoy Chakraborty. Compression laws for large language models. *arXiv preprint arXiv:2504.04342*, 2025.
- Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.
- Tunyu Zhang, Xinxi Zhang, Ligong Han, Haizhou Shi, Xiaoxiao He, Zhuowei Li, Hao Wang, Kai Xu, Akash Srivastava, Hao Wang, Vladimir Pavlovic, and Dimitris N. Metaxas. T3D: Few-step diffusion language models via trajectory self-distillation with direct discriminative optimization. *arXiv preprint arXiv:2602.12262*, 2026.
- Siyan Zhao, Zhihui Xie, Zichen Liu, Kaiwen Huang, Tianyu Pang, Changyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

A Additional Related Work

Block-diffusion language models and few-step decoding. Masked diffusion language models [Sahoo et al., 2024, Nie et al., 2025] and block-diffusion hybrids [Arriola et al., 2025, Cheng et al., 2025, Wang et al., 2025] trade sequential depth for parallel width. Training-free accelerators such as Fast-dLLM [Wu et al., 2025] push the same trade-off at inference time. T3D [Zhang et al., 2026] attributes few-step degradation to factorization error in the token-independent reverse process, which grows as steps shrink. Our work differs in scope: we do not propose an accelerator. We characterize the shape of the failure at small scale, per sample rather than in the mean, and quantify how post-training moves it.

336 **Self-distillation for fewer steps.** SDTT [Deschenaux and Gülgehre, 2025] distills a many-step
337 teacher into a few-step student. T3D [Zhang et al., 2026] distills on rollout trajectories produced
338 under the target decoding procedure. OPSD [Zhao et al., 2026] makes distillation on-policy for
339 autoregressive models: teacher and student share weights, the teacher conditions on privileged
340 information, and both are evaluated on the student’s own rollout. A concurrent work adapts on-policy
341 self-distillation to diffusion models on LLaDA-8B [Luo et al., 2026]. We differ from all of these
342 in what we measure. Prior work reports post-distillation accuracy on students that produce usable
343 trajectories. We measure the precondition itself: whether a small student near the floor produces
344 trainable trajectories at all, and what controls that.

345 B Additional Figures

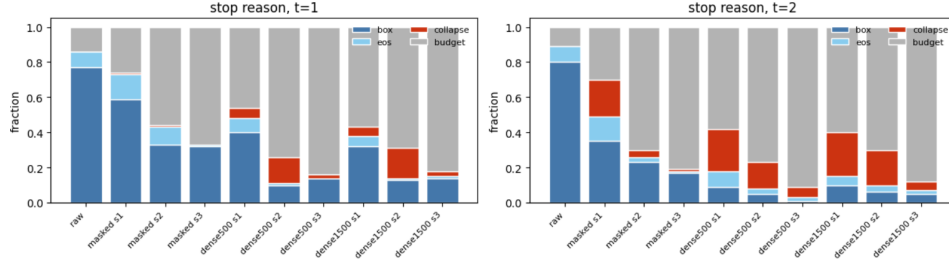


Figure 5: Stop reason for every generation on the 100-problem benchmark at $t = 1$ (left) and $t = 2$ (right). The base model ends most generations with a boxed answer. Every fine-tuned checkpoint ends most generations by exhausting the 1,024-token budget without a boxed answer.

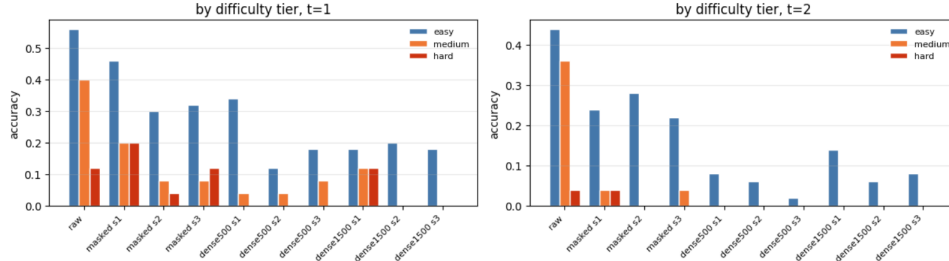


Figure 6: Accuracy by difficulty tier on the 100-problem benchmark (50 easy, 25 medium, 25 hard). At $t = 2$ no dense checkpoint solves any medium or hard problem.

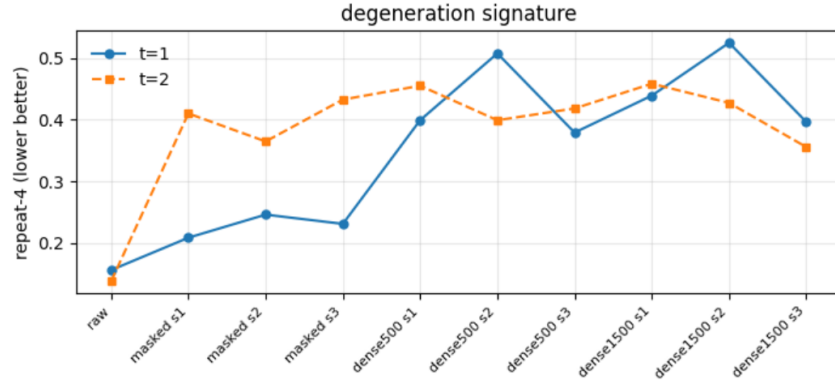


Figure 7: Mean repeat-4 per checkpoint on the 100-problem benchmark. Commit-only checkpoints stay near the base model at $t = 1$; dense checkpoints do not.

C Hyperparameters and Training Details

C.1 SFT Training

We use the same base model and LoRA configuration for both the dense (supervise-all) and commit-only SFT arms. Unless otherwise stated, all hyperparameters are kept identical between the two arms. The main difference is the supervision objective: the dense arm supervises every currently masked position (L_{all}), while the commit-only arm supervises only the top- t positions committed at that step (L_{commit}).

Table 4: Hyperparameters used for the Dense₅₀₀, Dense₁₅₀₀, and commit-only SFT arms.

Hyperparameter	Dense ₅₀₀	Dense ₁₅₀₀	commit-only
Base model	SDAR-4B-Chat-b32	SDAR-4B-Chat-b32	SDAR-4B-Chat-b32
Block size	32	32	32
LoRA rank	32	32	32
LoRA α	64	64	64
LoRA dropout	0.05	0.05	0.05
Optimizer	AdamW	AdamW	AdamW
Learning rate	1×10^{-5}	1×10^{-5}	1×10^{-5}
LR schedule	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	10	10	10
Epochs per stage	2	2	2
Number of stages	3	3	3
Batch size	1	1	1
Gradient accumulation	4	4	4
Effective batch size	4	4	4
Maximum target tokens	384	384	384
GPU	Nvidia RTX Pro (96 GB)	Nvidia RTX Pro (96 GB)	Nvidia RTX Pro (96 GB)
Data schedule	500	1500	1500
Loss function	L_{all}	L_{all}	L_{commit}

The three SFT stages use the same overall training setup, with the curriculum progressing through $t = 4$, $t = 2$, and $t = 1$ tokens per step. Each stage is trained for two epochs. The effective batch size is four sequences, obtained from a per-step batch size of one and gradient accumulation over four steps.

The dense and commit-only arms were implemented with the same optimizer, learning rate, scheduler, LoRA configuration, batch mechanics, and target length. Dense₁₅₀₀ and Commit-only share pools and preprocessing and differ only in which positions contribute to the loss. This controlled setup allows us to compare the effect of supervision density, while the differences in data sourcing and target preprocessing described in Section 4.2 remain important confounds.

C.2 Compute and Runtime

All our experiments, including the SFT and screening runs, were performed on a single Google Colab RTX Pro GPU for approximately 23 hours per experimental training and 12 hours for screening and benchmarking. No distributed data parallelism or model sharding was used in these experiments.

All experiments use a single GPU. The GPU model for SFT experiments were run on a Nvidia RTX Pro 6000 GPU. Runtime differences therefore reflect both the computational workload and the available hardware.

D Statistical Tests

Confidence intervals on the 100-problem benchmark are 95% percentile bootstrap intervals over problems with 10,000 resamples. Comparisons between two checkpoints use a two-sided two-proportion z -test with a pooled standard error on the per-problem correct/incorrect outcomes. Pooled comparisons sum correct counts over the three curriculum stages (300 rollouts per arm) and apply the same test. Because the same 100 problems appear in every stage, the pooled test overstates the effective sample size, and we treat the per-stage tests as the more conservative evidence.

E Assets and Licenses

Assets and licenses. Table 5 lists every external asset we use. All are publicly available, and our use of each falls within its stated terms.

Table 5: External assets used in this work.

Asset	Role	License
SDAR-4B-Chat [Cheng et al., 2025]	Primary model under study	Apache 2.0
TraDo-4B [Wang et al., 2025]	Replication model (Section 4.3)	MIT
DeepMath-103K [He et al., 2025]	Training and evaluation data	MIT

All three licenses permit research use, fine-tuning, and reporting of derived results. We redistribute none of these assets; we release only our evaluation code and the fixed 40-problem benchmark subset for Section 4.1 and 100-problem benchmark subset for Section 4.2 (problem indices into the public DeepMath-103K dataset, not the underlying text itself).

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the main findings of the paper and limit the claims to the experiments performed. In particular, we do not claim a final benefit from self-distillation because distillation training is not completed in this work.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 discusses the single 40-problem benchmark, single random seed, use of LoRA on one 4B model, differences between the SFT arms, target truncation, compute limitations, and the fact that final self-distillation performance is not yet measured.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.

- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present new theoretical results, theorems, or formal proofs. Its conclusions are based on empirical experiments and analysis.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper and appendix provide the model, benchmark size, decoding schedules, training procedure, LoRA configuration, optimizer, learning rate, batch mechanics, target length, compute setup, and screening settings needed to reproduce the reported experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often

one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The paper provides a mirror anonymous repository linked in the abstract, containing the code and data required to reproduce the main experimental results. The repository includes instructions for running the experiments and reproducing the reported results, together with the experimental details provided in the paper and appendix.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper describes the fixed evaluation benchmark, decoding schedules, training curriculum, supervision objectives, LoRA settings, optimizer, learning rate, batch size, gradient accumulation, target length, and compute resources. Additional hyperparameters and training details are provided in Appendix-C (C.1).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report 95% bootstrap confidence intervals and two-sided two-proportion z -tests for the 100-problem benchmark; statistical procedures and limitations are detailed in Appendix D. For the 40-problem benchmark, we report approximate binomial uncertainty of ± 15 percentage points.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix-C (C.2) reports the GPU type, available GPU memory, number of GPUs, and approximate runtime for the main experimental runs. The SFT experiments used one Nvidia RTX Pro 6000 GPU for approximately 23 hours.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work is computational research on language models and mathematical reasoning datasets and does not involve human subjects, personal data collection, or experiments that introduce identifiable risks to participants. We have followed standard research and attribution practices.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The work is foundational research on the reliability and efficiency of diffusion language models. Positive impacts include better understanding of failure modes in efficient language generation, while the main potential negative impact is that improved generative models could be misused; our work does not introduce a new deployment or application with a direct high-risk use.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not introduce or release a new high-risk model, image generator, or scraped dataset. It studies existing language models and mathematical data for research purposes and does not require additional release safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We use three existing assets, each cited at first use in the paper: SDAR-4B-Chat [Cheng et al., 2025] (Apache 2.0), TraDo-4B [Wang et al., 2025] (MIT), and DeepMath-103K [He et al., 2025] (MIT). All three licenses permit research use, fine-tuning, and reporting of derived results, and we redistribute none of the original assets. A full table of assets, roles, and licenses is provided in Appendix E.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce a new dataset, pretrained model, or other standalone research asset. The contribution is an empirical analysis of existing diffusion language models and training procedures.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.

- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing, human participants, participant instructions, or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects or participants are involved in this research, so IRB approval or equivalent human-subject review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The language models studied in this paper are the objects of the experiments, not tools used to develop the methodology or perform the research analysis. Any LLM assistance used for writing or editing does not affect the core methodology, experimental design, or scientific conclusions.

Guidelines:

736
737
738
739

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.